1. Array

```
#include<iostream>
using namespace std;
class Array
{
      int arr[10];
      int item,pos,N,loc;
   public:
      void Insert();
      void Delete();
      void Traverse();
      void GetData();
      void Search();
   Array()
   {
      N=0;
   }
   };
void Array::GetData()
{
   int n;
   cout<<"Enter how many element:(<10)";
   cin>>n;
   N=n;
   cout<<"enter"<<n<<"element:";
   for(int i=0;i<n;i++)
   cin>>arr[i];
}
void Array::Insert()
{
   cout<<"\n Enter the item to be inserted :";
```

```
   cin>>item;
   cout<<"\n Enter the Position : (<"<<N<<").";
   cin>>pos;
         for(int i=N-1;i>=pos-1;i--)
             arr[i+1]=arr[i];
          arr[pos-1]=item;
          N=N+1;
}
void Array::Delete()
{
   cout<<"\n Enter the Position of the Item to be deleted: ";
   cin>>pos;

   if (pos>N && pos<=0)
   {
      cout<<"\n Invalid Position ";
     return;
   }
   else
   {
      item=arr[pos-1];
        for(int i=pos-1;i<N;i++)
           arr[i]=arr[i+1];
     cout<<"\n Deleted item is : <<item";
     N=N-1;
   }
}
   void Array::Search()
{
   loc=-1;
   cout<<"\nEnter the item to be Search :";
   cin>>item;
```

```
   for(int i=0;i<N && loc==-1;i++)
   if(arr[i]==item)
      {
         loc=i+1;
      }
      if(loc==-1)
         cout<<"\n Item is not present in the list";
      else
         cout<<"\n Item is present in the list at location"<<loc<<".";
}
void Array::Traverse()
{
   cout<<"\n List of item in the list :\n";
   for(int i=0;i<N;i++)
      cout<<"\t"<<arr[i];
}
int main()
{
   Array obj;
   obj.GetData();
   int ch;
   do
   {

cout<<"\n********MENU********\n1.Insert\n2.Delete\n3.Traverse\n4.Search\
n5.Exit";
      cout<<"Enter Your Choice : ";
      cin>>ch;

         switch(ch)
         {
            case 1:obj.Insert();break;
```

```
            case 2:obj.Delete();break;
            case 3:obj.Traverse();break;
            case 4:obj.Search();break;
            case 5:break;
            default:cout<<"\nInvalid Choice . ";
          }
    }while(ch!=5);
  return 0;
}
```

2. bubble

```
#include <iostream>
using namespace std;

class Bubble
{
   int a[100], n;

public:
   void getData()
   {
      cout << "Enter number of elements: ";
      cin >> n;

      cout << "Enter elements:\n";
      for (int i = 0; i < n; i++)
      {
         cin >> a[i];
      }
   }

   void sort()
   {
      for (int i = 0; i < n - 1; i++)
      {
         for (int j = 0; j < n - 1 - i; j++)
         {
            if (a[j] > a[j + 1])
            {
               int temp = a[j];
               a[j] = a[j + 1];
               a[j + 1] = temp;
```

```
            }
          }
        }
    }

    void display()
    {
        cout << "Sorted elements are:\n";
        for (int i = 0; i < n; i++)
        {
            cout << a[i] << " ";
        }
        cout << endl;
    }
};

int main()
{
    Bubble b;

    b.getData();

    cout << "\nBefore Sorting:\n";
    b.display();

    b.sort();

    cout << "\nAfter Sorting:\n";
    b.display();

    return 0;
}
```

3. Selection

```
#include<iostream>
using namespace std;

class selectionsort
{
public:
  int i, *a, n, loc;

  selectionsort()
  {
    cout << "\n enter size of array:";
    cin >> n;
    a = new int[n];
  }

  void getdata()
  {
    cout << "\n enter " << n << " element:";
    for (int i = 0; i < n; i++)
      cin >> a[i];
  }

  void putdata()
  {
    for (int i = 0; i < n; i++)
      cout << " " << a[i];
  }

  void selection()
  {
    for (i = 0; i < n - 1; i++)
```

```
    {
      loc = i;
      for (int j = i + 1; j < n; j++)
      {
        if (a[j] < a[loc])
        {
          loc = j;
        }
      }

      if (loc != i)
      {
        int temp = a[loc];
        a[loc] = a[i];
        a[i] = temp;
      }
    }
  }
};
int main()
{
  selectionsort s;
  s.getdata();

  cout << "\n before sorting:";
  s.putdata();
   s.selection();
  cout << "\n after sorting:";
  s.putdata();
  return 0;
}
```

4. Insertion

```
#include <iostream.h>
#include <conio.h>

class insertionsort
{
public:
  int *a, n;

  insertionsort()
  {
    cout << "\nEnter the size of array:";
    cin >> n;
    a = new int[n];
  }

  void getdata()
  {
    cout << "\nEnter " << n << " element:";
    for(int i = 0; i < n; i++)
      cin >> a[i];
  }

  void putdata()
  {
    for(int i = 0; i < n; i++)
      cout << " " << a[i];
  }

  void insertion()
```

```
  {
    int i, j, key;

    for(i = 1; i < n; i++)
    {
      key = a[i];
      j = i - 1;

      while(j >= 0 && a[j] > key)
      {
        a[j + 1] = a[j];
        j--;
      }

      a[j + 1] = key;
    }
  }
};

void main()
{
  clrscr();

  insertionsort i;

  i.getdata();

  cout << "\nBefore Sorting the elements are:";
  i.putdata();

  i.insertion();
```

```
    cout << "\nAfter Sorting the elements are:";
    i.putdata();

    getch();
}
```

5. Merge

```
#include <iostream>
#include <conio.h>
using namespace std;

class Mergesort {
public:
  int i, j, *A, n;

  Mergesort(int size) {
    n = size;
    A = new int[n + 1];
  }

  void getdata() {
    cout << "\nEnter the number of elements: ";
    for (i = 1; i <= n; i++) {
      cin >> A[i];
    }
  }

  void putdata() {
    cout << "\nElements are: ";
    for (i = 1; i <= n; i++) {
      cout << A[i] << " ";
    }
  }

  void merge_sort(int low, int high) {
    int mid;
    if (low < high) {
```

```
    mid = (low + high) / 2;
    merge_sort(low, mid);
    merge_sort(mid + 1, high);
    merge(low, mid, high);
  }
}

void merge(int low, int mid, int high) {
  int h, i, j;
  int *B = new int[n + 1];
  h = low;
  i = low;
  j = mid + 1;

  while (h <= mid && j <= high) {
    if (A[h] <= A[j]) {
      B[i] = A[h];
      h++;
    } else {
      B[i] = A[j];
      j++;
    }
    i++;
  }

  if (h > mid) {
    for (int k = j; k <= high; k++) {
      B[i] = A[k];
      i++;
    }
  } else {
    for (int k = h; k <= mid; k++) {
```

```
            B[i] = A[k];
            i++;
          }
        }

        for (int k = low; k <= high; k++) {
          A[k] = B[k];
        }

        delete[] B;
    }
};

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    Mergesort m(n);
    m.getdata();

    cout << "\nBefore Sorting";
    m.putdata();

    m.merge_sort(1, n);

    cout << "\nAfter Sorting";
    m.putdata();

    getch();
    return 0;
}
```

6. Quick

```
#include <iostream.h>
#include <conio.h>

class Quicksort
{
   public:
      int *a;
      int n;

      Quicksort()
      {
         cout << "\nEnter the size of array:";
         cin >> n;
         a = new int[n];
      }

      void getdata()
      {
         cout << "\nEnter " << n << " elements:";
         for (int i = 0; i < n; i++)
            cin >> a[i];
      }

      void putdata()
      {
         cout << "\nElement before sorting are:";
         for (int i = 0; i < n; i++)
            cout << " " << a[i];
      }
```

```
void sort(int low, int high)
{
   int j;
   if (low < high)
   {
      j = partition(low, high);
      sort(low, j - 1);
      sort(j + 1, high);
   }
}

int partition(int low, int high)
{
   int pivot, temp, i, j;
   pivot = a[high];
   i = low;
   j = low;

   while (i <= j)
   {
      if (a[i] > pivot)
      {
         i++;
      }
      else
      {
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
         i++;
         j++;
```

```
            }
          }
          return (j - 1);
       }
};

void main()
{
   clrscr();

   Quicksort q;
   q.getdata();

   cout << "\nThe element before sorting are:";
   q.putdata();

   q.sort(0, q.n - 1);

   cout << "\nThe sorted element are:";
   q.putdata();

   getch();
}
```

7. linear search

```
#include <iostream>
using namespace std;

class Lsearch
{
public:
  int a[10], i, n, no, loc, k;

  Search()
  {
    loc = -1;
  }

  void getdata()
  {
    cout << "\nEnter the no of elements:";
    cin >> k;

    cout << "\nEnter the elements:";
    for (i = 0; i < k; i++)
    {
      cin >> a[i];
    }
  }

  int search()
  {
    int item;
    getdata();
```

```
      cout << "\nEnter the element to be searched:";
      cin >> item;

      int i;

      for (i = 0; i < k; i++)
      {
         if (a[i] == item)
         {
            return i;
         }
      }

      return -1;
   }
};

int main()
{
   int loc = -1;

   Lsearch l;
   loc = l.Lsearch();

   if (loc == -1)
      cout << "\nThe given element is not present.";
   else
      cout << "\nThe given element is present at location:" << loc + 1;

   return 0;
}
```

8. binary search

```
#include <iostream>
using namespace std;

class bsearch
{
public:
  int a[10], lb, ub;

  bsearch()
  {
    lb = 0;
    ub = 4;
  }

  void getdata()
  {
    cout << "\nEnter the five elements in the ascending order:";
    for (int i = lb; i <= ub; i++)
    {
      cin >> a[i];
    }
  }

  void binary()
  {
    int beg, end, mid, item;

    cout << "\nEnter the element to search:";
    cin >> item;
```

```
    beg = lb;
    end = ub;
    mid = (beg + end) / 2;

    while (beg <= end)
    {
      if (a[mid] < item)
        beg = mid + 1;

      else if (a[mid] == item)
      {
        cout << "\nThe number " << item
          << " found at position " << mid + 1;
        break;
      }

      else
        end = mid - 1;

      mid = (beg + end) / 2;
    }

    if (beg > end)
    {
      cout << "\nThe number " << item
        << " is not found in given array";
    }
  }
};

int main()
```

```
{
   bsearch b;

   b.getdata();
   b.binary();

   return 0;
}
```